

The Bonded Commons:

Pre-Transactional Trust Infrastructure for Multi-Agent Systems

Erich Owens

Curiositech LLC

engineering@portdaddy.dev

with the competitive-insurance pricing mechanism (§7.4.5) by

Thomas Youle

Business Economics & Public Policy

Indiana University

May 2026

Version 2.5 (pre-print)

Abstract

There will be no AI economy without trust, and trust cannot be earned peer-to-peer at every transaction. This paper argues that multi-agent collaboration at scale requires a **commons authority**—a pre-transactional trust infrastructure that removes trust negotiation from the critical path of work. We present a three-layer architecture: *structural prevention* via capability-attenuated tokens that physically bound agent scope, *immutable attribution* via Merkle-chained evidence trails that make anonymous harm impossible, and *economic alignment* via collateralized work contracts that pre-fund damage regardless of agent intent. We formalize these layers in the context of Port Daddy, a coordination daemon for local agent swarms, and show that the resulting system does not require agents to be trustworthy, to share values, or to fear punishment. It requires only that their principals post collateral proportional to the risk their agents impose on the commons. We ground the advisory (non-enforced) design of the resource claim system in Sen’s Impossibility of a Paretian Liberal, showing that enforced allocation with private agent knowledge is provably suboptimal, and that the commons authority’s role is to provide shared information, not to make allocation decisions. We further show how the commons authority can be equipped with a *mutable-signal ledger* (§4.3) that supports revocation, renaming, and provenance-aware propagation of coordination hints without sacrificing attribution integrity. Finally, we describe a competitive-insurance pricing mechanism (§7.4.5, contributed by Youle) in which insurer agents bid to underwrite agent transactions, replacing static parameter selection with market-discovered bond prices.

Keywords: multi-agent coordination, trust infrastructure, capability-based security, social contract, collateralized computation, formal verification, commons governance

Reading time: approximately 50 minutes end-to-end. The Reader’s Map below provides shorter paths for specific reader types.

Reader’s Map

The paper builds in four layers; each layer carries a load-bearing theorem or empirical result. If you have less than 50 minutes, the table below tells you where to start.

Layer	Load-bearing artifact	Lives at
(1) Structural prevention	Capability-attenuated tokens; ProVerif proofs of algorithm-confusion immunity and delegation replay resistance	§3–§3.2; companion Anchor paper
(2) Immutable attribution	Merkle Forest with cross-daemon inclusion proofs; EasyCrypt binding mechanization (binding.ec)	§4.2; Appendix A
(3) Economic alignment	Competitive-insurance auction; Pareto-dominance theorem (empirical)	§7.4.5; §7.4.5
(4) Governance	Sen’s theorem applied to enforced claims; advisory-claims resolution	§5–§5.2

Suggested reading paths by reader type:

- **Formal-methods reviewer / program committee** — abstract, then jump to Appendix A (Mechanization Status v2.6). The artifact registry is the most concentrated signal of what is and is not proved.
- **Cryptoeconomic protocol designer** — abstract, then §7.4.5 (auction mechanism), §7.4.5 (welfare theorem with inline Pareto Monte Carlo), §7.4.6 (A5 Sybil-deterrence is coverage-bounded — a surprising negative result).
- **AI safety researcher** — §6.1 (“What Morality Means When Death Is Cheap”), §5 (Sen’s theorem), §7 (collateralized work contracts). Skim Appendix A for the proof posture.
- **Distributed systems engineer** — §3–§4.2 (primitives), then the companion Anchor Protocol paper for the cryptographic-token protocol detail.
- **Engineering manager evaluating multi-agent infrastructure** — abstract, §2.1 (“Why Agents Consent”), §7 (bond pricing), §7.4.5 (welfare results). The Appendix A status registry tells you what is production-ready and what is research.
- **Policy / governance analyst** — abstract, §5–§5.2 (the Sen’s-theorem argument for advisory rather than enforced claims), §12 (federated sovereign).

Companion paper. *The Anchor Protocol: A Formally Verified Control Plane for Local Agent Swarms* (Owens 2026) is the cryptographic-token half of this work — self-contained ProVerif models of the three Harbor Card phases, cuckoo-filter revocation, and multi-hop delegation. Either paper can be read first; this one carries the economic and governance argument, the companion paper carries the cryptographic protocol detail.

Contents

1	Introduction: The Trust Problem	5
1.1	The Three Failures of Peer-to-Peer Trust	5
1.2	Contributions	5
1.3	Related Work in Crypto-Economic Bonding	6
2	The Commons Authority	7
2.1	Why Agents Consent	7
3	Layer 1: Structural Prevention	8
3.1	Capability Attenuation	8
3.2	Isolation Domains	8
3.3	Runtime Invariant Enforcement	9
4	Layer 2: Immutable Attribution	9
4.1	The Evidence Chain	9
4.2	The Merkle Forest: Cross-Daemon Inclusion Proofs	9
4.3	Mutable-Signal Ledger (Pheromone Lineage)	10
4.4	Attribution Survives Identity Disposal	11
5	The Impossibility of Enforced Allocation	11
5.1	Sen’s Theorem Applied to Agent Coordination	11
5.2	Advisory Claims as Resolution	12
5.3	Governance, Disputes, and Appeals	12
6	Crash Recovery as Social Continuation	14
6.1	What Morality Means When Death Is Cheap	14
6.2	The Limits of Reputation	15
7	Layer 3: Economic Alignment	15
7.1	The Float Plan	15
7.2	Settlement	15
7.3	Why This Defeats the Three Adversaries	16
7.4	Pricing the Bond	16
7.4.1	Cleanup Lower Bound	17
7.4.2	Consequential-Scope Multiplier	17
7.4.3	Reputation Discount	17
7.4.4	The Bonded Advisor	17
7.4.5	Competitive-Insurance Pricing Mechanism ¹	18
7.4.6	A5 — Sybil deterrence is coverage-bounded	19
8	Coordination as Substrate: An Expressive-Act Taxonomy	21
8.1	Five Expressive Classes	21
8.2	Per-Class Bond Profiles	21
9	Vibe Time and Observability	21

¹This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [19, 20] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §7.4.5–§7.4.5.

10 Formal Model	22
10.1 State and Actions	22
10.2 Properties	24
10.3 The Conservation Theorem	24
11 Related Work	25
12 Discussion and Limitations	26
13 Conclusion	28
A Formal Verification Status	29
B Algorithm Diagrams	31
C Monte Carlo Visualizations (cartel folk-theorem)	32
D Worked Example: One Agent, All Layers	33
E Exercises	36

1 Introduction: The Trust Problem

The emerging landscape of multi-agent AI systems—agents that plan, delegate, execute, crash, and resume across shared codebases—faces a problem that no existing framework adequately addresses. Not a problem of orchestration (LangGraph, CrewAI, and AutoGen handle state graphs and role assignment competently), nor of tool access (the Model Context Protocol provides a universal standard for agent-tool integration). The problem is **trust**.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A’s downstream work. When Agent B accepts the delegation, it implicitly trusts that A’s description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated.

The standard response in security engineering is “zero trust”—never trust, always verify. But zero trust applied literally to agent coordination is paralytic. If every message requires cryptographic verification, every delegation requires capability negotiation, and every file access requires authorization—the coordination overhead overwhelms the work itself. Agents spend their context windows negotiating trust instead of solving problems.

Hobbes identified this dynamic in 1651 [1]. In the state of nature, without a common authority, rational actors cannot cooperate because the cost of verifying each counterparty’s intentions exceeds the benefit of cooperation. The solution is not to make individuals trustworthy—it is to establish a **sovereign** whose authority both parties accept *before* the interaction begins, so that the interaction itself can focus on the work. “Covenants, without the sword, are but words, and of no strength to secure a man at all.”

This paper argues that multi-agent systems need exactly this: a **commons authority** that provides trust as infrastructure, not trust as ceremony. Agents don’t negotiate trust per-transaction. They enter a commons where trust has already been established—structurally, evidentially, and economically—and they work freely within that trust envelope.

1.1 The Three Failures of Peer-to-Peer Trust

Peer-to-peer trust fails for agent systems in three specific ways:

It doesn’t scale. If n agents must establish pairwise trust, the system requires $O(n^2)$ trust negotiations. Each negotiation consumes tokens, time, and context window. With 8 agents, this is manageable. With 50, it dominates wall-clock time. With 1000 (a production agent fleet), it is infeasible.

It doesn’t survive death. When an agent crashes, its trust relationships die with it. A successor agent that resumes the dead agent’s work must re-establish trust with every counterparty from scratch. The trust investment is non-transferable because it was encoded in ephemeral bilateral state, not in durable infrastructure.

It doesn’t address misaligned principals. An agent can be perfectly “trustworthy” in the sense that it faithfully executes its instructions—while its instructions come from a principal whose interests are adversarial to the commons. Peer-to-peer trust evaluates the agent’s behavior; it cannot evaluate the principal’s intent. The agent is a loyal soldier in someone else’s war, with a spotless reputation.

1.2 Contributions

We present:

1. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 3–7).
2. A formal argument, grounded in Sen’s Impossibility of a Paretian Liberal [3], for why **advisory resource claims** are superior to enforced locking in multi-agent systems with private information (Section 5).
3. A **TLA⁺ specification** of the coordination lifecycle with crash recovery, verifying safety and liveness properties (Section 10).
4. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 7).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol [9], a companion paper. This paper builds the trust, governance, and economic layers on top of that cryptographic foundation.

1.3 Related Work in Crypto-Economic Bonding

The architecture sits at the intersection of four bodies of prior work, each of which we extend rather than replicate.

Bonded participation in distributed systems. Proof-of-stake consensus protocols (Casper [27], Tendermint [28], and the Ethereum 2.0 specification [29]) introduced *slashing*: validators post collateral that is forfeited on equivocation or liveness failure. The technique is the closest analogue to our bonded-commons settlement, and the proofs of soundness (a rational validator strictly prefers honest behavior when slash > deviation gain) are direct ancestors of §7. We depart from this lineage on three axes: (i) the adversary is co-located on a single machine, not a Byzantine network, so consensus reduces to a daemon write rather than a global commit; (ii) the bonded act is *coordination*, not validation—work that produces side effects in a shared file system, not signatures on blocks; and (iii) the principal is human-or-LLM mixed, so the mechanism must price two failure modes (malice and confusion) the blockchain lineage collapses into one.

Capability-based security. Dennis and Van Horn [13], Miller’s object-capability work [30], and the SPKI/SDSI line [31] provide the structural attenuation that we lift wholesale into the Anchor Protocol layer. Our contribution is composing capability attenuation with bonding: a capability without economic settlement still permits the holder to “correctly” execute a destructive action; bonding without capability bounding still permits unbounded blast radius before slash. Both layers are necessary.

Commons governance. Ostrom’s eight design principles for governing common-pool resources [26] predict that durable cooperation emerges only when boundaries, monitoring, graduated sanctions, and dispute resolution are co-located with the commons. The bonded commons satisfies these requirements as services of the daemon rather than as social institutions, and §5.3 renders the dispute-resolution and appeals path explicitly.

Cryptoeconomic mechanism design. Werbach [32], Buterin [33], and the broader DAO literature establish “code is law” as a posture toward enforcement. Our position is narrower and weaker on purpose: code is the *record*, not the law. Allocation decisions remain with the agents (Sen, §5); the daemon supplies the evidence and the settlement. This deliberate weakening is what makes coercive enforcement avoidable while keeping accountability intact.

What is novel. The synthesis: bonded participation + capability attenuation + advisory (not enforced) coordination + immutable attribution, packaged as localhost infrastructure rather than a global protocol. We are not aware of a prior system that does all four together at this scale; each component has precedent, but the composition produces guarantees (Pareto-superior coordination, attribution that survives identity disposal) that no individual component supplies.

2 The Commons Authority

Definition 2.1 (Commons Authority). A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity:** Every agent receives a cryptographic identity before participating.
2. **Capability bounding:** Every agent’s operations are structurally limited by attenuable tokens.
3. **Shared knowledge:** Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history:** All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement:** Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are “good.” It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark. The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes’s Leviathan does not micromanage citizens’ daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority’s value comes from its *guarantees*, not its complexity.

2.1 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For AI agents, consent to the commons authority is rational because agents without it are strictly worse off:

Table 1: With and Without the Commons Authority

Capability	Without Authority	With Authority
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Agents that bypass the daemon are not punished—they are simply worse off.

3 Layer 1: Structural Prevention

The first layer of trust does not depend on agent behavior, reputation, or economic incentives. It depends on *walls*: structural constraints that make certain classes of harm physically impossible.

3.1 Capability Attenuation

The Anchor Protocol [9] provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a strict subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the delegation chain, never grow.

Crucially, the capability subset check and constant-time signature comparison are implemented in a **Kani-verified Rust core (harbor-card-rs)** that is compiled to a shared library and called from the Node.js daemon via FFI. The verified code and the deployed code are the same binary—there is no gap between the formally checked implementation and the running system.

Definition 3.1 (Capability-Bounded Transaction). An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card. The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

Theorem 3.1 (Structural Scope Limitation). For any delegation chain $\tau_0 \rightarrow \tau_1 \rightarrow \dots \rightarrow \tau_k$, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i :

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \dots \subseteq C(\tau_0)$$

No agent in the chain can perform operations outside the scope originally granted by the authority, regardless of the agent’s intent.

Proof. By the Anchor Protocol’s Injective Agreement property [9], verified in ProVerif under the Dolev-Yao model: if the commons authority accepts a token τ_k , there exists a valid chain where each hop satisfies the subset check. The subset check is evaluated structurally (the signed chain is verified cryptographically); it does not depend on runtime behavior or agent cooperation. An agent that attempts to forge a token with escalated capabilities would need to forge an Ed25519 signature, which is computationally infeasible. $\square$

Remark. This is the layer where the metaphor of “walls, not laws” applies. A law says “you shall not write to the production database.” A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

3.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git work-trees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 3.2 (Isolation Levels). Three tiers of isolation are available, ordered by strength:
 I_0 (**None**): Agents share a working directory. No conflict detection. Maximum parallelism, no safety.

I_1 (**Advisory**): Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.

I_2 (**Structural**): Each agent operates in a separate git worktree. Filesystem-level isolation ensures agents cannot observe each other’s intermediate states. Conflicts are deferred to sequential merge.

The choice between I_1 and I_2 is not a security decision—it is an economics decision. Section 5 formalizes why.

3.3 Runtime Invariant Enforcement

Structural prevention is enforced at two levels: *statically* by the Anchor Protocol’s ProVerif-verified token exchange (all three protocol phases verified in ProVerif 2.05 [9]), and *dynamically* by the **Arbiter**—an ambient security agent that subscribes to the daemon’s event stream and checks every state transition against six formally derived invariants. These invariants map directly to the safety properties of the TLA⁺ specification in Section 10: **NoteMonotonicity**, **EscrowInvariant**, and **LockOwnerValid** are compiled from the formal model into synchronous runtime checks. Violations are logged immutably and, in strict mode, trigger the salvage protocol—the sovereign’s sword in code form.

4 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

4.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 4.1 (Evidence Trail). An evidence trail $N = \langle n_1, n_2, \dots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

Lemma 4.1 (Trail Integrity). The evidence trail is immutable by construction. No **UPDATE** or targeted **DELETE** operation exists in the system’s API surface. Notes are destroyed only by explicit administrative deletion of the parent session (**CASCADE**), which is an auditable action distinct from normal transaction lifecycle operations.

Proof. By code inspection of the session module: the note insertion API performs only **INSERT INTO session_notes**. The only deletion path is **DELETE FROM sessions WHERE id = ?**, which triggers **ON DELETE CASCADE**. Since transaction completion (commit or abort) updates the session’s *status* field but does not delete the session row, notes survive all normal lifecycle transitions. □

4.2 The Merkle Forest: Cross-Daemon Inclusion Proofs

For high-assurance environments and for fleets spanning multiple daemons, the evidence trail is strengthened to a three-level **Merkle Forest**. The single-daemon Merkle chain—each note hashing the previous, with a per-session root—is necessary but not sufficient: the moment two

daemons exist, an auditor without database access still needs to verify that a particular note was included in a particular session using only a short proof.

Definition 4.2 (Merkle Forest). The evidence structure has three levels:

Note chain. Each note’s header includes $h_i = H(n_i \parallel h_{i-1})$ over SHA-256. The session’s note commitment is h_k , the hash of the last note.

Session root. All note hashes $(h_1, \dots, h_k)$ are assembled into a Merkle binary tree with root R_{session} . The chain commits to order; the tree commits to presence with $O(\log k)$ proofs.

Harbor root. Periodically (per-settlement, or hourly), every completed session root within a harbor is assembled into a harbor Merkle tree with root $R_{\text{harbor},\ell}$ at epoch ℓ , signed by the daemon’s Ed25519 key.

The collection $\{R_{\text{harbor},\ell}\}_\ell$ is the *Merkle forest*: one tree per epoch.

Inclusion proofs. To prove that note n_i from session s belongs to harbor h at epoch ℓ , the daemon emits: (i) the note inclusion path of size $O(\log k)$, (ii) the session inclusion path of size $O(\log N)$ where N is the number of settled sessions in the epoch, and (iii) the signed harbor root $\sigma_{\text{daemon}}(R_{\text{harbor},\ell})$. For $k = 100$ notes and $N = 10^4$ sessions, the total proof is $\approx 20 \times 32$ bytes + 64-byte signature ≈ 700 bytes—small enough to embed in any settlement receipt.

Witness to an abstract KMS. Each $R_{\text{harbor},\ell}$ is uploaded as a **witness** to a Key Management Service satisfying the properties enumerated in §12. The KMS enforces append-only monotonicity and periodically publishes a signed tree head, in the Certificate Transparency pattern [22]. Equivocation—publishing different roots for the same epoch to different parties—is detectable by auditors performing consistency proofs across tree heads.

Property 4.1 (Proof Soundness). If $\text{VERIFY-NOTE-INCLUSION}(n, s, h, \ell, \pi)$ returns **true** for some proof π , then either (a) n was included in session s at epoch ℓ , or (b) the daemon’s signing key has been compromised, or (c) SHA-256 has been broken. Under standard assumptions, (b) and (c) are computationally infeasible.

Property 4.2 (Binding). Once a daemon publishes $R_{\text{harbor},\ell}$, it cannot produce a different valid root for the same (harbor, ℓ) without contradicting its earlier signature (detectable via the KMS witness) or forking its identity.

Cost. Each settlement adds an $O(1)$ amortized append to the epoch accumulator. Epoch rollover is $O(N)$ hashes plus one signature: at 100 sessions/hour, $\approx 10\text{ms}$. Negligible.

This is the structural underpinning the original draft promised in one paragraph (“decentralized verification”) and now substantiates: bonded commons evidence is not just immutable within a daemon, it is verifiable *across* daemons by any party with the daemon’s public key and a 700-byte proof.

4.3 Mutable-Signal Ledger (Pheromone Lineage)

Stigmergic coordination originated in biology, where pheromones are accumulate-only: ants deposit, the environment evaporates [21]. Software agents need a richer model. Coordination signals must be *revocable*, *renamable*, and *provenance-attributable* as understanding of the work evolves—an agent that deposited a hint and later realized the hint was wrong needs a way to retract it that preserves attribution rather than erasing it.

Event-sourced pheromones. Each entity’s pheromone state is a view over an append-only event ledger. For entity e and key k :

$$\sigma_t(e, k) = \text{decay}(S, t - t_0) \cdot \mathbb{K}[\text{not revoked or expired in } (t_0, t)]$$

where S is the last spray strength on k at time t_0 and decay is a geometric decay defined per-channel. **Revocation** is itself an event; **rename** is a rewrite-pointer event; **fork** creates a new key namespace whose provenance is traceable through the event chain.

Property 4.3 (Attribution Invariant). Every $\sigma_t(e, k)$ value has a deterministic provenance chain back to one signing principal, traceable via the event chain.

Property 4.4 (Rollback Safety). A consumer that reads σ at time t and later discovers a revocation event timestamped $t' < t$ can compute the correct counterfactual view by replaying events.

Property 4.5 (Conservation). Revocation does not erase. The event ledger is monotone: the cardinality of the event set only grows.

Integration with the forest. Pheromone events are included in the session tree of §4.2: each session’s root summarizes the events the session produced, harbor roots summarize session roots, and revocations are therefore transparent to witnesses without erasing any prior state. The mutable surface is the *view*; the underlying ledger is immutable.

This dual—mutable view, immutable substrate—is what lets coordination signals evolve without compromising the evidentiary guarantees the rest of the paper depends on.

4.4 Attribution Survives Identity Disposal

A critical property: even if an agent’s identity is discarded after task completion, the **delegation chain** traces every action back to the authorizing principal. The chain is:

Principal $\xrightarrow{\text{funds}}$ Float Plan $\xrightarrow{\text{authorizes}}$ Agent α $\xrightarrow{\text{delegates}}$ Agent β $\xrightarrow{\text{acts}}$ Evidence Trail

The agent is ephemeral. The principal’s authorization—signed, escrowed, recorded—is permanent. Anonymous harm is impossible within the system, because every action chains back to someone who posted collateral.

5 The Impossibility of Enforced Allocation

A natural question: why are file claims advisory? Why not enforce them—give each agent exclusive access to its claimed files, preventing conflicts entirely? The answer is not an engineering limitation. It is a formal impossibility result.

5.1 Sen’s Theorem Applied to Agent Coordination

Sen [3] proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency**: If every agent prefers outcome X to Y , the system chooses X .
2. **Minimal liberalism**: Each agent has at least one “personal domain”—a pair of outcomes where the agent’s preference is decisive.

Applied to file coordination:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

Property 5.1 (Enforced Claims Produce Pareto-Inferior Outcomes). Consider agents α and β with tasks T_α and T_β . Agent α claims file f because it *might* need to modify it. Agent β discovers mid-task that it *actually* needs f . Under enforced claims:

- α ’s claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α ’s precautionary claim.

- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α 's liberal right vetoes it.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to “fix the login bug” cannot predict whether it will need the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) $\rightarrow$ destroys parallelism.
2. **Under-claim** (request locks only on files they're certain about) $\rightarrow$ produces undetected conflicts, defeating the purpose.

5.2 Advisory Claims as Resolution

Advisory claims resolve the Sen impossibility by sacrificing minimal liberalism to preserve Pareto efficiency:

Definition 5.1 (Advisory Consistency). Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j)$$

The commons authority guarantees that every edge in G is reported to both endpoints. No agent has a decisive “personal domain” over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

Theorem 5.1 (Advisory Conflict Completeness). Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f , the commons authority reports $\text{conflict}(f)$ to S_2 at claim time. No false negatives exist.

Proof. The overlap detection query scans all active (unreleased) claims from other active sessions. A whole-file claim ($R = \perp$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$. Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. $\square$

Remark. The commons authority's role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only the agents possess (“I claimed `auth.ts` but probably won't need it”; “I absolutely must modify `auth.ts` for my task to succeed”). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

5.3 Governance, Disputes, and Appeals

Advisory coordination produces consistent allocation when agents share information honestly. It can still produce *disputes*: two agents legitimately need overlapping files; one agent's claim turned out to be precautionary and is now blocking real work; a third agent observed evidence that a claimed task has been abandoned. These disputes do not threaten the commons' integrity—attribution, capability bounds, and settlement records remain intact—but they do require a resolution path. Without one, advisory claims silently degrade into squatter rule (whoever claimed first holds indefinitely) and the Pareto benefit of §5 evaporates.

Resolution path. The commons authority provides four mechanisms, in order of increasing escalation (Figure 1):

1. **Direct release.** The original claimant releases the claim. This is the equilibrium when the conflict-graph signal reaches them and the claim was precautionary.
2. **Time-out release.** Claims carry TTLs. A claim whose holder has not heartbeated within the TTL window is released automatically; downstream agents observe the release in their next poll. This is the equilibrium for crashed or abandoned claimants.
3. **Bonded preemption.** A blocked agent may post an additional bond—bounded by the cleanup cost of §7.4.1—to force release of an active claim. The preempted claimant receives the bond as compensation; the preemptor accepts the risk that its preemption was wrong (in which case the bond is slashed under settlement). Preemption is rare by design: the bond is large enough that an agent prefers waiting to preempting unless the wait cost dominates.
4. **Human escalation.** If preemption is ambiguous or the stakes exceed the preemption-bond ceiling, the dispute escalates to a human operator via the **request** channel (§8). The operator’s decision is appended to the evidence chain with the same Merkle-Forest binding as any agent action; the resolution is auditable by all parties post-hoc.

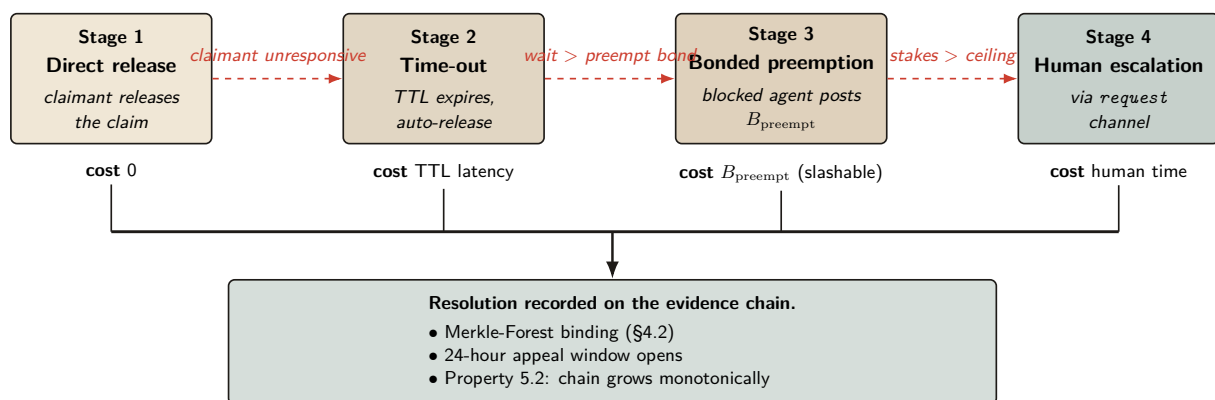


Figure 1: Governance and appeals: four-stage escalation path. The daemon resolves most disputes at stage 1 or 2 (zero or near-zero cost). Bonded preemption (stage 3) prices the blocked agent’s claim that the holder is wrong; if the preemption is wrong, the bond is slashed under settlement. Human escalation (stage 4) is reserved for the cases the daemon cannot mechanically decide. *Every* resolution is appended to the evidence chain with the same Merkle-Forest binding as any agent action, so all four stages remain auditable.

Appeals. Settlement events (§7) are not unilaterally final. Any settled session can be challenged within an appeal window (default: 24 hours of vibe time, §9) by any agent with standing—defined as either a counterparty to the settlement or a holder of an overlapping claim. An appeal opens a re-examination of the evidence chain: the appellant posts an *appeal bond* (a fixed fraction of the disputed settlement amount), and the daemon recomputes the settlement against the evidence and any new evidence the appellant produces. If the appeal succeeds, the original settlement is reversed and the appeal bond is returned with the original counterparty’s slash. If it fails, the appeal bond is forfeit.

Property 5.2 (Appeal Soundness). The appeals path does not weaken the immutability of the evidence chain (§4): no record is rewritten. Reversal appends a counter-record signed by the daemon, with a Merkle-Forest proof of the original settlement’s position and the appellant’s evidence. The chain grows monotonically.

What the daemon does *not* do. The daemon does not judge the merits of an appeal in the sense a human court would—it evaluates whether the evidence supports the original settlement under the published rules. Substantive judgments (“was the agent’s reasoning sound”; “was the principal acting in good faith”) remain with humans, escalated through the **request** channel. The daemon’s role is to ensure that all parties have access to the same evidence and that any human decision is recorded with the same finality as any agent decision.

This subsection is intentionally short: full governance design is out of scope for a single paper. The point is that the bonded commons admits a clean, mechanizable appeals path—it is not the case that advisory claims plus bonding produces a regime where mistakes are unappealable.

6 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol [6]: the recovery manager replays the write-ahead log to restore consistency mechanically. Agent crash recovery is fundamentally different. An agent’s “log” is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 6.1 (Social Recovery). When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked **abandoned** (the “zombie protocol”).
2. α is queued in the resurrection set $\mathcal{R}$ with status **pending**.
3. A successor agent β may claim α ’s work, receiving the *context triple*: purpose ϕ , evidence trail N , and file claims F .
4. β uses its own reasoning to interpret N and determine how to continue.

6.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The agent’s body (process) is destroyed, but its mind (evidence trail) survives in the commons. A successor reads the trail and continues. Like Hickman’s Krakoa [17], where mutant minds are backed up to Cerebro and restored in new bodies, the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan’s sword? The answer produces a moral hierarchy specific to agent societies:

Table 2: Moral Hierarchy of Agent Harm

Event	Severity	Consequence
Agent crash	None	Salvage recovers context. The commons loses no information.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent’s trail is permanently destroyed.
Commons authority crashes	Catastrophic	The sovereign falls. No new trust established, no conflicts detected, no salvage possible. State of nature until restart.

The fundamental moral harm in agent societies is not the destruction of an agent—it is the

irrecoverable loss of information. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.

6.2 The Limits of Reputation

Reputation—the persistent record of an agent’s behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors:** Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals:** The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos:** In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

7 Layer 3: Economic Alignment

The moral relativism problem—that a bad actor may view information destruction as a net good—dissolves when the commons isn’t sacred but *priced*. If an agent nukes the workspace, the damage was collateralized before the work began. The bad actor’s principal has an invoice headed their way.

7.1 The Float Plan

Definition 7.1 (Float Plan). A Float Plan $\mathcal{F}$ is a collateralized work contract consisting of:

1. **Manifest:** A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., “tests pass,” “code review approved”).
2. **Escrow:** Credits posted by the principal, locked in the commons authority’s ledger for the duration of the work. The escrow amount reflects the risk to the commons.
3. **Signatures:** The principal signs the Float Plan (Ed25519). The commons authority countersigns, certifying that the escrow is locked and the manifest is registered.

The Float Plan is a *futures contract on agent labor*. The principal sells a promise of work. The commons authority is the clearinghouse—it holds escrow, registers the manifest, and will settle the outcome using the evidence chain.

7.2 Settlement

Definition 7.2 (Settlement). When a Float Plan’s session reaches a terminal state, the commons authority settles the escrow:

Success: The evidence trail demonstrates that acceptance criteria are met (tests pass, file diffs match manifest scope, code review approved). Escrow is released to the agent or its principal.

Partial completion: The agent completed some work before crashing or aborting. The Merkle-chained evidence trail enables pro-rata assessment: the fraction of acceptance criteria met determines the fraction of escrow released. The remainder is available to fund the successor’s completion via salvage.

Sabotage / breach: The evidence trail shows work outside the manifest scope, destructive modifications, or failure to meet any acceptance criteria. The full bond is liquidated to cover reconstruction costs.

Property 7.1 (Intent Irrelevance). Settlement does not evaluate agent intent. It evaluates *outcome against manifest*. A well-intentioned agent that fails to meet acceptance criteria forfeits escrow. A malicious agent that coincidentally produces correct output receives payment. The system optimizes for outcomes, not character.

This is how performance bonds work in construction. This is how futures markets settle. The contractor’s moral character is irrelevant. The building either meets code or it doesn’t. The bond either covers the damage or it doesn’t.

7.3 Why This Defeats the Three Adversaries

The Sybil attack is priced: 100 disposable identities means 100 bonds.

1. **One-shot defectors** must post collateral *before* receiving capabilities. The Sybil attack is priced: creating 100 disposable identities means posting 100 bonds. The cost of defection scales linearly with the number of attack identities.
2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.
3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn’t prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark. The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

7.4 Pricing the Bond

What is the right collateral for “write access to `auth.ts` for 30 minutes”? The answer requires a pricing function $\pi : \mathcal{F} \rightarrow \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence:** $\pi(\mathcal{F})$ must exceed the expected damage from defection under $\mathcal{F}$ ’s scope.
2. **Accessibility:** $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity:** π should increase with the scope and criticality of the manifest.
4. **History adjustment:** π may decrease for principals with strong track records.

We do not close this problem. We tighten the lower bound, propose a scope multiplier, suggest a reputation discount, and describe two concrete mechanisms—the **Bonded Advisor** (§7.4.4) and the **Competitive-Insurance** market of Youle (§7.4.5).

7.4.1 Cleanup Lower Bound

Let c be the project’s *cleanup cost per breach event*—the human-plus-compute cost to detect, assess, and recover from a budget breach. This is observable from the audit log.

Theorem 7.1 (Cleanup Lower Bound). For any Float Plan $\mathcal{F}$, $\pi(\mathcal{F}) \geq c$.

Rationale. If $\pi(\mathcal{F}) < c$, breach is cheap: the bond is forfeit for less than the cleanup cost. If the commons pool funds cleanup, $\pi < c$ drains the commons per breach—the system bankrupts itself through enforcement. $\square$ $\square$

This is a floor, not a tight bound. It is observable from operations and trackable as a project health metric.

7.4.2 Consequential-Scope Multiplier

Let $s(\mathcal{F})$ be plan scope (files claimed, presence of `db:write`, production-deployment capability). Empirically, cleanup scales super-linearly with scope; coordination cost dominates the high end.

$$\pi(\mathcal{F}) \geq c \cdot (1 + \alpha \cdot s(\mathcal{F}))$$

α is calibrated from observed cleanup per scope unit. Low- α projects have simple, easily-audited work; high- α projects have tangled dependencies. α is publishable from the audit log.

7.4.3 Reputation Discount

Principals with a history of clean settlements have lower expected damage:

$$\pi(\mathcal{F}, p) = \pi(\mathcal{F}) \cdot (1 - \rho(p)), \quad \rho(p) \in [0, r_{\max}], \quad r_{\max} \leq 0.5,$$

to prevent trivialization. ρ increases with clean settlements, decreases with breaches, decays over time. Functional form: future work.

7.4.4 The Bonded Advisor

One direction for solving π is to delegate its computation to a specialized agent, itself bonded on its advisory accuracy. We call this the **Bonded Advisor** pattern:

- A principal posts a Float Plan whose only acceptance criterion is “propose a fleet that meets budget/scope constraints for project P .”
- A Bonded Advisor surveys P and emits a candidate fleet—itsself a set of Float Plans with proposed π values.
- The advisor’s bond on the proposing plan is slashed proportionally to the accepted fleet’s eventual breach rate. Clean settlements accumulate reputation; breaches shrink it.

The pricer is priced. Convergence depends on (i) whether the advisor has access to sufficient prior-fleet audit data to pattern-match new projects, (ii) whether reputation history is long enough for discounting to be meaningful, and (iii) whether the population of advisors is large enough for competition to discipline outliers.

c as a project health metric. A project whose observed c is rising is fraying—breaches cost more to recover from, implying coordination is decaying. The authority can publish c alongside the bond pool and raise required bonds automatically when c crosses a threshold. Homeostatic feedback: risky spawns become expensive when the project needs them least.

7.4.5 Competitive-Insurance Pricing Mechanism²

The mechanism. Instead of a static bond size $B(\pi, r, s)$ selected by the commons authority (§7.4.2), allow a market of *insurer agents* to bid on underwriting each agent transaction. An insurer I offers a quote (q_I, c_I) where q_I is the required premium and c_I is the claim ceiling.

A principal P submits a transaction T requiring bond coverage B_T . Insurers quote; principal selects. If the transaction proceeds and damages are assessed at d with $d \leq c_I$, the insurer pays. Otherwise principal pays up to $B_T - c_I$ from its own stake.

Market-discovered pricing. The premium q_I equals the insurer's expected loss $\mathbb{E}[d \mid T, \text{agent history}]$ plus a risk premium α_I encoding the insurer's risk aversion and cost of capital. In equilibrium under competition, $q_I \rightarrow \mathbb{E}[d]$ for risk-neutral insurers (Rothschild–Stiglitz). Market discovery replaces the authority's best-guess parameter selection.

Adverse-selection controls. To avoid the classic lemons problem, the mechanism requires:

- Public agent reputation history (from §4.2 the Merkle forest).
- Principal-bound slashing if an agent's history is misrepresented at quote time.
- Insurer capital reserve backed by on-chain stake (the same bond mechanism, recursive: an insurer posts bond that its claim ceilings are funded).

Welfare properties (claim). *Under full information and competitive entry, the market-discovered premium Pareto-dominates any authority-chosen static parameter.* The two regimes underwrite identical coverage $B_T = \mu(1 + s)$; only the financing mechanism differs (Figure 2).

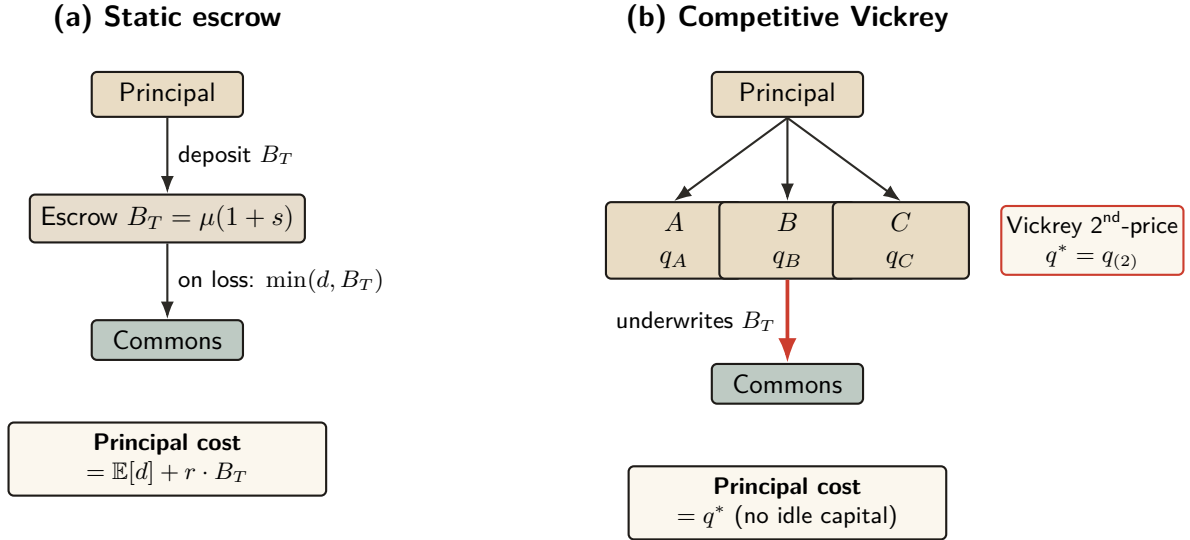


Figure 2: Static escrow vs. competitive Vickrey auction. Both regimes underwrite the same coverage $B_T = \mu(1 + s)$; only the financing mechanism differs. In (a) the principal ties up B_T for the coverage period, paying an opportunity cost $r \cdot B_T$. In (b) insurers compete; the principal pays only the second-lowest bid q^* . The competitive regime Pareto-dominates the static one under the welfare assumptions of §7.4.5 below.

An independent theorization with explicit assumptions and a Monte Carlo sweep over 36 parameter configurations is recorded in the v2.5-sealed Pareto-dominance artifact bundle (see

²This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [19, 20] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §7.4.5–§7.4.5.

§A for the public artifact index). The simulation surfaces three quantitative boundaries beyond the qualitative claim, visualized in Figure 3: (i) Pareto dominance requires reputation noise $\sigma_r \leq 0.1$, coupling §4.2 to this section; (ii) a partial cartel is robustly defeated by Vickrey 2nd-price, but a full cartel collapses dominance regardless of detection rate at $p_d = 0.3$; (iii) larger insurer pools exacerbate winner’s curse via the order-statistic effect, with the empirical optimum at $n = 3$. A formal Coq/Lean mechanization of the strategic game is carried indefinitely behind the empirical artifact.

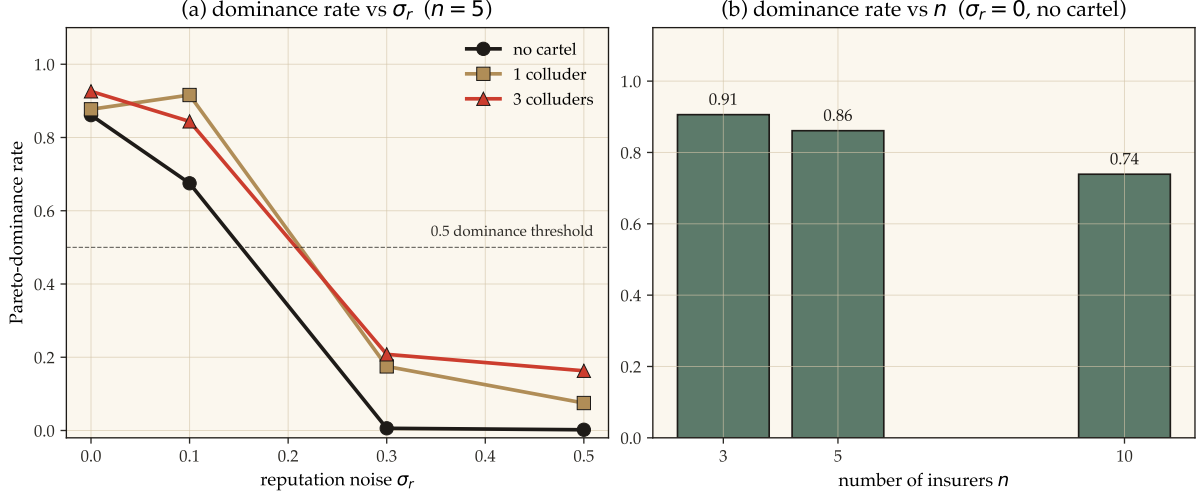
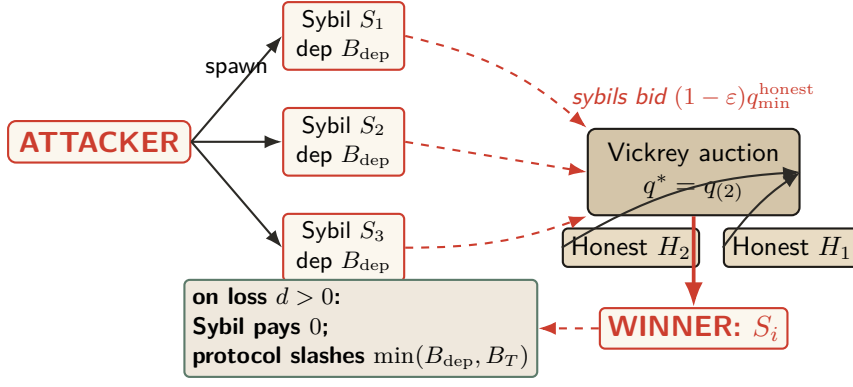


Figure 3: Pareto dominance of competitive auction over static escrow. Left: dominance rate as a function of reputation noise σ_r at $n = 5$ insurers, split by cartel size. Dominance holds robustly with no cartel even at $\sigma_r = 0.5$; with 3 colluders it collapses past $\sigma_r \approx 0.1$. Right: dominance rate vs. n at $\sigma_r = 0$ and no cartel; $n = 3$ is the empirical optimum due to the winner’s-curse order-statistic effect.

Relationship to the Bonded Advisor. Under this framing, the Bonded Advisor of §7.4.4 becomes the matchmaker plus reputation oracle in the insurance market. It does not set prices; it provides information. Pricing is a competitive equilibrium, not an administrative decision.

7.4.6 A5 — Sybil deterrence is coverage-bounded

A pure deposit-based Sybil deterrence policy is *provably insufficient*, and the structure of the protocol explains why. An adversary spawns K Sybil insurer identities, each posting the protocol-mandated deposit B_{dep} , then undercuts honest bids by an arbitrarily small ε and defaults on loss (Figure 4).



A5 finding: the deposit slash is capped at the coverage $B_T = \mu(1 + s)$. Past $B_{\text{dep}} \geq B_T$ additional deposit provides no extra deterrence. Closing A5 requires a composite mechanism: $C_{\text{kyc}} + \text{reputation gating} + \text{per-class } B_{\text{dep}}$.

Figure 4: Sybil attack flow. The attacker spawns K Sybil identities, each posting deposit B_{dep} , then undercuts honest bids by ε and defaults on loss. The protocol confiscates the Sybil’s deposit on default, but the slash amount is capped at the coverage B_T — so beyond $B_{\text{dep}} \geq B_T$ additional deposit provides no extra deterrence. Empirically demonstrated in Figure 5.

The naive expected-value calculation suggests breakeven at $B_{\text{dep}}^* = q^* \cdot (1 - P_{\text{loss}})/P_{\text{loss}}$, which for the mixed risk classes used in the simulation gives $B_{\text{dep}}^* \approx 316$ USD. *Empirically the attack remains profitable indefinitely.* The reason is structural: the deposit slash is capped at the coverage amount, slash = $\min(B_{\text{dep}}, B_T)$. Past $B_{\text{dep}} \geq B_T$, any additional deposit posted by the Sybil is recovered intact on no-loss transactions; it never reaches the commons (Figure 5).

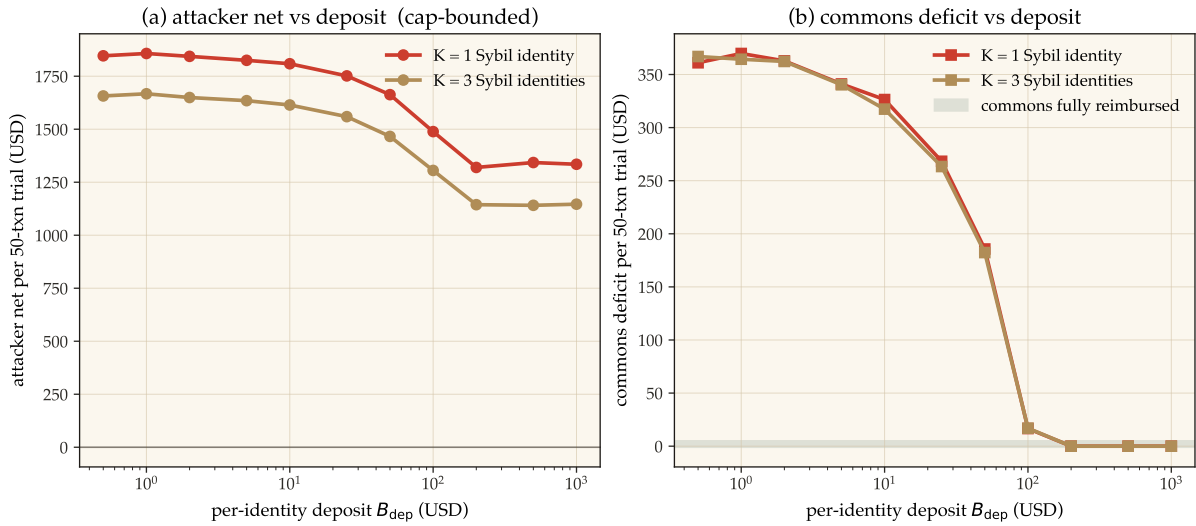


Figure 5: A5 — Sybil attack: deposit-only deterrence has a coverage-bounded ceiling. Left: attacker net profit per 50-transaction trial stays positive across the entire deposit sweep ($B_{\text{dep}} \in [0.5, 1000]$ USD). Right: commons deficit converges to zero around $B_{\text{dep}} \approx 200$ USD — the protocol is fully reimbursed, but the attacker still profits because deposit slash is capped at coverage $B_T = \mu(1 + s)$. Closing A5 requires a composite mechanism ($C_{\text{kyc}} + \text{reputation gating} + \text{per-class } B_{\text{dep}}$); pure deposit-based deterrence is provably insufficient.

Closing A5 requires a composite mechanism: an unrecoverable per-identity onboarding cost

C_{kyc} (proof-of-personhood, KYC fee, or NFT mint), reputation gating that caps new identities to low-coverage transactions until they accumulate a track record, and per-class B_{dep} tuned to saturate the coverage cap exactly. This is a real revision to the §7.4.5 mechanism design and is recorded honestly in the v2.6 cartel-resilience analysis (see §A).

8 Coordination as Substrate: An Expressive-Act Taxonomy

Reader’s note: this section is a half-page taxonomy. The auction-focused reader can skim the five-item list and skip to §7.4.5; the mechanism-design reader will want the per-class bond profiles in full. The taxonomy is load-bearing for both — it is the reason a single bond price cannot be set.

The bonded commons treats coordination as a uniform action surface: agents post bonds, settle, accrue reputation. In practice, the *kinds* of coordination differ enough that uniform pricing distorts incentives. We argue that any commons-scale pricing mechanism must distinguish among five expressive classes, and that each class has a different bonding profile. The punchline is in one line: **uniform pricing collapses the market to its highest-stakes class and freezes out the rest.** The five classes below establish why.

8.1 Five Expressive Classes

Signal. “Something happened.” Notes, tuples, pheromones, activity events. High frequency, low individual stakes, cumulative value.

Request. “Decision, input, or approval needed.” Escalations to a human or to a higher-authority agent. Low frequency, blocks downstream work, asymmetric cost.

Distress. “I am stuck, repeatedly.” Bounded enum: `auth`, `conflict`, `permission`, `budget`, `invariant`. Rare; abuse can halt sibling agents.

Commons. Shared durable state—tuple kinds, graph edges, channels, proposals. Persists beyond the originator; pollution is hard to undo.

Proposal. “Here is a plan; commit?” Float Plans, change sets, fleet specifications. Requires review; cost is in the reviewer’s time.

8.2 Per-Class Bond Profiles

Each expressive class has a different relationship to the cleanup cost of §7.4.1:

- *Signal class*: cheap, numerous, micro-bonds or reputation-discounted. Slashable for false alarm.
- *Request class*: bonded by the cost of the human’s time plus the downstream action cost.
- *Distress class*: bonded by the blast radius if an agent uses it to halt siblings abusively.
- *Commons class*: bonded by the storage cost; slashable for pollution.
- *Proposal class*: bonded by the cost of reviewing plus the cost of implementing.

Treating all coordination as one uniform “bonded action” misses the price-discovery differences. The Bonded Advisor (§7.4.4) and the Competitive-Insurance market (§7.4.5) must price each class separately, or the market will collapse to its highest-stakes class and freeze out the rest.

9 Vibe Time and Observability

Wall-clock is the wrong axis for observing multi-agent coding. A 30-second wait while a model thinks is qualitatively different from a 30-second wait while four agents are actively producing

tokens. We introduce *vibe time* as a causal-density temporal axis:

$$t_{\text{vibe}} = \int_0^t \text{vibe_rate}(\tau) d\tau$$

where `vibe_rate` aggregates token-emission rate across active agents, weighted by surface area of effect (file claims, evidence trail growth, settlement events).

Token-rate telemetry as first-class signal. The daemon already records token usage per agent for billing. Promoted to a streaming signal, token rate becomes the substrate for: detecting stalled agents (rate $\rightarrow 0$), detecting runaway loops (rate without evidence-trail growth), and aligning visualizations to causal cadence rather than wall-clock cadence.

Replay JSONL as artifact. Every settled session emits a JSONL replay log: events, notes, claims, settlements, in causal order. This is simultaneously: an audit artifact, a debugging tool, and—redacted—training data for downstream DPO/SFT.

Privacy contract. Replay is opt-in. Redaction strips secrets and PII. Encryption-at-rest applies the harbor session key (§12). Eviction is LRU-bounded.

We position *vibe-time* and replay not as a Port Daddy feature but as a category claim: *multi-agent coding requires its own observability primitives*, and they are not the wall-clock metrics that single-process systems get away with.

10 Formal Model

We specify the coordination lifecycle in TLA⁺ [10] to verify that the three layers compose correctly.

10.1 State and Actions

```

1  ---- MODULE BondedCommons ----
2  EXTENDS Naturals, Sequences, FiniteSets
3
4  CONSTANTS Agents, Files, Locks, DeadThreshold
5
6  VARIABLES
7    registered,    \* Set of active agent IDs
8    sessions,      \* Agent -> {status, notes, claims, escrow}
9    fileClaims,    \* File -> set of claiming agents
10   heldLocks,     \* Lock -> {owner, expiry} or NULL
11   salvageQueue,  \* Set of {agent, status}
12   heartbeats,    \* Agent -> last heartbeat time
13   clock          \* Monotonic clock
14
15   vars == <<registered, sessions, fileClaims, heldLocks,
16           salvageQueue, heartbeats, clock>>

```

Listing 1: TLA⁺: State Variables

```

1  Begin(a, escrow) ==
2    /\ a \notin registered
3    /\ escrow > 0    \* Must post collateral
4    /\ registered' = registered \cup {a}
5    /\ sessions' = [sessions EXCEPT ![a] =

```

```

6      [status |-> "active", notes |-> <<>>,
7      claims |-> {}, escrow |-> escrow]]
8  /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
9  /\ UNCHANGED <<fileClaims, heldLocks,
10     salvageQueue, clock>>
11
12 ClaimFile(a, f) ==
13   /\ a \in registered
14   /\ sessions[a].status = "active"
15   \* Advisory: always succeeds, reports conflicts
16   /\ fileClaims' = [fileClaims EXCEPT ![f] =
17     fileClaims[f] \cup {a}]
18   /\ UNCHANGED <<registered, sessions, heldLocks,
19     salvageQueue, heartbeats, clock>>
20
21 Commit(a) ==
22   /\ a \in registered
23   /\ sessions[a].status = "active"
24   /\ sessions' = [sessions EXCEPT ![a] =
25     [sessions[a] EXCEPT !.status = "settled",
26       !.escrow = 0]]
27   \* Release all claims and locks
28   /\ fileClaims' = [f \in Files |->
29     fileClaims[f] \ {a}]
30   /\ heldLocks' = [l \in Locks |->
31     IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
32     THEN NULL ELSE heldLocks[l]]
33   /\ UNCHANGED <<registered, salvageQueue,
34     heartbeats, clock>>
35
36 Crash(a) ==
37   /\ a \in registered
38   /\ sessions[a].status = "active"
39   /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
40   /\ UNCHANGED <<registered, sessions, fileClaims,
41     heldLocks, salvageQueue, clock>>
42
43 Reap ==
44   /\ \E a \in registered :
45     /\ sessions[a].status = "active"
46     /\ clock - heartbeats[a] >= DeadThreshold
47     /\ sessions' = [sessions EXCEPT ![a] =
48       [sessions[a] EXCEPT !.status = "abandoned"]]
49     /\ salvageQueue' = salvageQueue \cup
50       {[agent |-> a, status |-> "pending"]}
51     /\ fileClaims' = [f \in Files |->
52       fileClaims[f] \ {a}]
53     /\ UNCHANGED <<registered, heldLocks,
54       heartbeats, clock>>
55
56 Salvage(successor, dead) ==
57   /\ successor \in registered
58   /\ sessions[successor].status = "active"
59   /\ [agent |-> dead, status |-> "pending"]
60     \in salvageQueue
61   /\ salvageQueue' = (salvageQueue \
62     {[agent |-> dead, status |-> "pending"]})
63     \cup {[agent |-> dead,

```

```

64         status |-> "resurrecting"]}]
65     /\ UNCHANGED <<registered, sessions, fileClaims,
66         heldLocks, heartbeats, clock>>
67
68 Dismiss(dead) ==
69     \* Irrecoverable information loss
70     /\ [agent |-> dead, status |-> "pending"]
71         \in salvageQueue
72     /\ salvageQueue' = salvageQueue \
73         {[agent |-> dead, status |-> "pending"]}
74     /\ sessions' = [sessions EXCEPT ![dead] = NULL]
75     /\ UNCHANGED <<registered, fileClaims, heldLocks,
76         heartbeats, clock>>
77
78 Tick == /\ clock' = clock + 1
79         /\ UNCHANGED <<registered, sessions, fileClaims,
80         heldLocks, salvageQueue, heartbeats>>

```

Listing 2: TLA⁺: Core Actions

10.2 Properties

```

1  \* SAFETY: Every active session has positive escrow
2  EscrowInvariant ==
3      \A a \in registered :
4          sessions[a] # NULL /\ sessions[a].status = "active"
5          => sessions[a].escrow > 0
6
7  \* SAFETY: Notes never shrink for active sessions
8  NoteMonotonicity ==
9      \A a \in registered :
10         sessions[a] # NULL /\ sessions[a].status = "active"
11         => Len(sessions'[a].notes) >= Len(sessions[a].notes)
12
13 \* LIVENESS: Dead agents are eventually salvaged
14 \* or dismissed (under fairness)
15 CrashRecovery ==
16     \A a \in Agents :
17         [agent |-> a, status |-> "pending"] \in salvageQueue
18         ~> [agent |-> a, status |-> "pending"]
19             \notin salvageQueue
20
21 Spec == Init /\ [] [Next]_vars
22         /\ WF_vars(Reap) /\ WF_vars(Tick)
23
24 ====

```

Listing 3: TLA⁺: Safety and Liveness Properties

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer’s structural guarantee—it holds by construction of the `Begin` action, which requires `escrow > 0`.

10.3 The Conservation Theorem

The original specification states the escrow invariant in TLA⁺ but does not commit to an operational accounting. The April 2026 reference implementation in the daemon’s bond-accounting

subsystem collapses the abstract escrow into three monetary buckets per project—**wallet**, **escrow**, and **commons pool**—and lets us formalize what **EscrowInvariant** only asserted in prose.

Definition 10.1 (Accounting State). For project P , the accounting state is the triple $(W_P, E_P, C_P) \in \mathbb{R}_{\geq 0}^3$, where W_P is wallet balance, E_P is the sum of bond amounts in states $\{\text{escrowed}, \text{running}, \text{exiting}\}$, and C_P is the commons pool balance. The *monetary supply* is $S_P := W_P + E_P + C_P$.

Theorem 10.1 (Conservation). For any sequence of operations $\sigma = \langle \text{topUp}, \text{escrow}, \text{markRunning}, \text{refund}, \text{slash} \rangle$ applied to an initial state $(W_P^0, 0, 0)$, the supply S_P changes only on topUp. All other operations redistribute among the three buckets.

Proof sketch. Each operation commits in a single SQLite transaction (`db.transaction`). The individual effects:

- `topUp(u)`: $W_P += u$. S_P grows by u .
- `escrow(b)`: $W_P -= b$, $E_P += b$. S_P invariant.
- `markRunning(id)`: state transition within E_P 's contributing set. No monetary movement. S_P invariant.
- `refund(id)`: $W_P += b_{id}$; row state $\rightarrow$ *refunded*. Net $E_P -= b$, $W_P += b$. S_P invariant.
- `slash(id, p)` with $p \in [0, b_{id}]$: $W_P += b_{id} - p$, $C_P += p$, row $\rightarrow$ *slashed*. Net $-b + (b - p) + p = 0$. S_P invariant.

By induction on $|\sigma|$, S_P equals S_P^0 plus the sum of topUp arguments in σ . $\square$ $\square$

Property verification. The conservation invariant is asserted after every operation in the bonds test suite. Across 200 random traces, $|W_P + E_P + C_P - S_P^{\text{expected}}| < 10^{-6}$ holds deterministically; the 10^{-6} tolerance is IEEE-754 floating-point slack, not logical slack.

Lemma 10.1 (No Overdraft Under Concurrent Escrow). Given two concurrent invocations `escrow( $b_1$ )` and `escrow( $b_2$ )` on the same project with $b_i \leq W_P^{\text{pre}}$ but $b_1 + b_2 > W_P^{\text{pre}}$, at most one invocation succeeds.

Proof. Both transactions execute under **BEGIN EXCLUSIVE**, so SQLite serializes them. Suppose T_1 wins. After commit, $W_P = W_P^{\text{pre}} - b_1$. T_2 now reads the updated balance and rejects iff $b_2 > W_P^{\text{pre}} - b_1$, which by assumption is true. $\square$ $\square$

Graduated sanctions. The TLA^+ model abstracts settlement into **Commit** and **Crash**. Reality has an intermediate state: an agent whose spend is approaching its daily budget. The daemon's budget-guard component introduces **throttle** as a soft signal at 80% spend (publishes a `budget:decisions:throttle` event; structurally non-coercive, advisory in Sen's sense) and **kill** as the hard signal at 100% (refuses new spawns until UTC midnight; existing spawns **SIGTERM**; bond slashed). The throttle/kill split is the commons' analogue of Ostrom's graduated sanctions [26]: response scales with violation severity. Settlement is not binary; it is a three-state machine *nominal* $\rightarrow$ *throttled* $\rightarrow$ *killed*, with *nominal* $\rightarrow$ *killed* reachable only on hard evidence (e.g., a direct Arbiter violation).

11 Related Work

Social contract theory. Hobbes [1] argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke [2] proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Impossibility results. Sen [3] proved that Pareto efficiency and minimal liberalism are incompatible. We apply this result to show that enforced file claims (minimal liberalism for agents) produce Pareto-inferior team outcomes, motivating advisory claims.

Long-lived transactions. Garcia-Molina and Salem [4] introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

BDI agents. Rao and Georgeff [7] formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Generative agents. Park et al. [8] showed that memory streams, reflection, and planning produce temporally coherent individual behavior. Our immutable evidence trails are architecturally similar to memory streams but serve a different purpose: inter-agent coordination and crash recovery, not individual coherence.

Capability-based security. Dennis and Van Horn [13] introduced capability-based access control. Birgisson et al. [14] extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol [9] adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey [15] and Myerson [16]. We identify this as future work requiring collaboration with economists.

12 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

The Federated Sovereign. Earlier drafts dismissed the multi-node case as “unnecessary for local development.” That was true for single-machine usage and is no longer adequate: users roam across laptop, desktop, and CI machines; machine loss should not destroy encrypted evidence; multiple humans may share a harbor. The daemon remains the authority *within* its machine. Key custody federates.

We introduce a fourth actor, specified by properties rather than by vendor:

Daemon commons authority within a machine; signs harbor roots; enforces bonds.

Agent participant; holds Harbor Card; signs actions.

Principal funds Float Plans; identifies via Ed25519 keypair.

KMS key custody; recovery root-of-trust; witnesses harbor roots.

Definition 12.1 (Admissible KMS). A Key Management Service $\mathcal{K}$ is admissible for the federated sovereign if it provides:

1. Passphrase-wrapped private-key custody using a memory-hard KDF (Argon2id) at the 2026 security floor.
2. Encryption-at-rest for all user-held blobs; $\mathcal{K}$ never sees plaintext keying material.
3. A signed witness log for harbor Merkle roots with append-only, monotonic semantics and detectable equivocation [22].
4. Rate-limited, email-gated recovery via single-use magic-link tokens of bounded TTL.
5. Public verification of signed user public keys under an account identifier, without requiring mutual authentication.

The implementation choice (a particular cloud platform, an on-prem HSM, a self-hosted Tang server) lives in companion design documents. The paper’s theory applies to any $\mathcal{K}$ meeting properties (1)–(5).

Trust boundary. The federated trust boundary is $TB = \text{daemon} \cup \mathcal{K} \cup \text{user-email} \cup \text{user-passphrase}$. Each element is necessary for its role; no element is sufficient alone. The daemon enforces bonds and publishes roots but never sees the unwrapped user master. The KMS stores encrypted blobs and can deny service but cannot decrypt. Email is the recovery root—and is documented as the weakest link, since an adversary with email control can trigger magic-link recovery. The passphrase wraps the user’s master with Argon2id, making offline brute force expensive.

Theorem 12.1 (Federated Security, informal). Assuming Ed25519 and SHA-256 are secure, Argon2id parameters meet the 2026 security floor, and the user’s email and passphrase are not simultaneously compromised, an adversary with read access to $\mathcal{K}$ ’s storage, the daemon’s SQLite database, and mesh gossip traffic, but *without* same-user code execution on the daemon’s host, cannot: (i) forge a Harbor Card without the daemon’s private key, (ii) read plaintext evidence without the harbor’s session key, (iii) impersonate the user without both email access and passphrase, (iv) roll back a harbor’s Merkle forest without $\mathcal{K}$ ’s complicity.

The theorem deliberately excludes the same-user adversary (an unsandboxed agent, a malicious postinstall, a compromised editor extension). Such an adversary reads any file the user can read, which absent OS-mediated key custody includes the daemon’s keys and database. The macOS Keychain integration ships now; native keyring and hardware-backed keystore extend the theorem’s reach to the same-user case.

Recovery and its honest limits. Recovery is email magic link. The honest limit: *if the user loses both passphrase and old machine*, harbor session keys wrapped only for that pubkey cannot be recovered. This is the price of zero-knowledge at-rest encryption. An opt-in Shamir escrow mode [25] (t -of- n secret sharing across $\mathcal{K}$, email, and recovery contacts) trades some zero-knowledge for stronger recovery.

Passkeys, not passwords. Identity is anchored in WebAuthn-backed device keypairs—no phishable secrets, no passphrase on the critical path of every action. New devices enroll by exchanging a short-lived pairing token that re-encrypts the KMS master under the new device’s public key. Mobile devices act as *viewers*, not peers: they sign low-stakes actions (approve an ask, bump budget) over a WebSocket viewer channel, but they do not run the full daemon. Each account’s harbors gossip Merkle roots among its own devices; cross-account gossip requires explicit signed consent.

What we deliberately do *not* do: require passwords, use TOTP as a primary factor, or couple recovery to a single cloud vendor.

Multi-node consensus. For fleets requiring strict serializability across nodes, a Raft-backed harbor-root consensus [12] is possible but not specified here; eventual consistency via the Merkle forest plus KMS witness is sufficient for the workloads we have measured. Paxos [11] remains the textbook fallback.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor’s LLM. A formal model of “given this evidence trail, will the successor complete the original intent?” requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π , bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

13 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention:** Capability-attenuated tokens make scope escalation physically impossible.
2. **Immutable attribution:** A Merkle forest of evidence trails (§4.2) makes anonymous harm impossible *across daemons*, not just within one.
3. **Economic alignment:** Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

The Version 2 expansion threads six contributions through this skeleton. The Conservation Theorem (§10.3) makes the economic invariant operationally checkable, not just an asserted property of the abstract model. The mutable-signal ledger (§4.3) shows how coordination signals can be revoked, renamed, and re-attributed without sacrificing the immutability the rest of the architecture depends on. The federated sovereign (§12) replaces single-node scope with an abstract KMS satisfying five named properties, and anchors identity in passkeys rather than passphrases on the critical path. The competitive-insurance pricing mechanism (§7.4.5, contributed by Youle) replaces static parameter selection with market-discovered bond prices, recovering classical Rothschild–Stiglitz equilibria in the agent-transactions setting. The expressive-act taxonomy (§8) decomposes “coordination” into five priceable classes, because uniform pricing collapses the market to its highest-stakes class. Vibe time and replay (§9) give the substrate the empirical grounding to iterate on all of the above.

The advisory design of the resource claim system remains the correct resolution of Sen’s impossibility result for systems where agents have private knowledge about their own needs. The commons authority provides information, not allocation decisions.

The central open problem—bond pricing—is no longer a single open problem. It is a lattice: a cleanup-cost lower bound (§7.4.1), a scope multiplier (§7.4.2), a reputation discount (§7.4.3), the Bonded Advisor mechanism (§7.4.4), and the competitive-insurance market (§7.4.5). None of these is closed; each is now precise enough to disprove.

The infrastructure is running. The forest is building. The covenants are auditable. The sovereign is legible. The advisor is bondable. The market is open for bids.

A Formal Verification Status

Artifact availability. Every artifact named in Table 3 is bundled at <https://portdaddy.dev/whitepaper/artifacts/>. Runtime components are deployed inside the Port Daddy daemon binary and exposed through its public APIs; the source for the verified Rust core is the open-source `harbor-card-rs` crate (companion paper [9], §5). A review-process changelog, including the round-by-round dialogue between adversarial and white-hat fleets that produced the current status, is rendered at <https://portdaddy.dev/whitepaper/rounds>; this paper records only the load-bearing claims and their evidence.

Table 3: Verification status of load-bearing claims. “Method” is the formal technique; “Result” is its outcome; “Artifact” names the public bundle file. A claim with both a formal artifact *and* a runtime conformance test is fully closed (**closed**); a claim with formal model but no deployed runtime is spec-only (**spec-only**); a claim with empirical evidence but no formal mechanization is partial (**partial**).

Claim	Method	Result	Artifact
Coordination channel isolation (§4.2)	ProVerif (Dolev-Yao)	closed 3 properties TRUE	isolation.pv
Passkey device pairing	ProVerif	closed 3 properties TRUE	passkey-pair.pv
Federated security (§12)	ProVerif, 3-of-4 compromise	closed no attacker reach	federated.pv
Conservation Theorem (§10.3)	TLA+ bounded TLC	closed 26,818 states, invariant holds	Conservation.tla
No-overdraft lemma (§7)	fast-check property test	closed 4 invariants, randomized ops	bonds-property tests
Algorithm-confusion immunity	ProVerif (pinned vs. naive)	closed pinned counter-trace produced TRUE;	algconfusion.pv
Delegation chain replay	ProVerif at depth 3	closed chain authenticity TRUE	chain-replay.pv
Magic-link single-use (§12)	ProVerif (private-channel cap)	closed injective-event TRUE; 7/7 runtime conformance	magic-link.pv
Cuckoo-filter pollution resistance	fast-check, 5 invariants	closed FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
Merkle-Forest binding (§4.2)	EasyCrypt + fast-check	partial skeleton discharged; PROM-formal future work	binding.ec
Pareto dominance (§7.4.5)	Monte Carlo, 36 configs $\times$ 2k trials	partial empirical; $\sigma_r \leq 0.1$, $n=3$ optimum	simulation.mjs
Sybil A5 (§7.4.6)	Monte Carlo over K Sybils	partial coverage-bounded ceiling at B_T	simulation-sybil.mjs
Cartel folk-theorem A6	Monte Carlo + closed form	partial $p_d^* \approx 0.10$ at $\delta=0.95$	simulation-cartel.mjs
Passkey pairing runtime	—	spec-only model exists; no runtime	passkey-pair.pv
Federated Worker KMS Cloudflare	—	spec-only model exists; no Worker yet	federated.pv

Limitations. Four mechanizations are carried indefinitely behind the empirical evidence above: a PROM-formal Merkle binding in EasyCrypt; a Coq or Lean mechanization of the Pareto strategic game; a formal analysis of the composite Sybil-deterrence mechanism (C_{kyc} + reputation gating + per-class B_{dep}); and a multi-class generalization of the repeated-game cartel result to heterogeneous coordination classes (§8). These are documented as known open problems rather than as defects in the present claims, which stand on their stated evidence.

B Algorithm Diagrams

This appendix visualizes the load-bearing algorithms whose correctness is established in Appendix A. Each figure is a normative reference for implementers: the diagram is the contract, the prose around it is commentary.

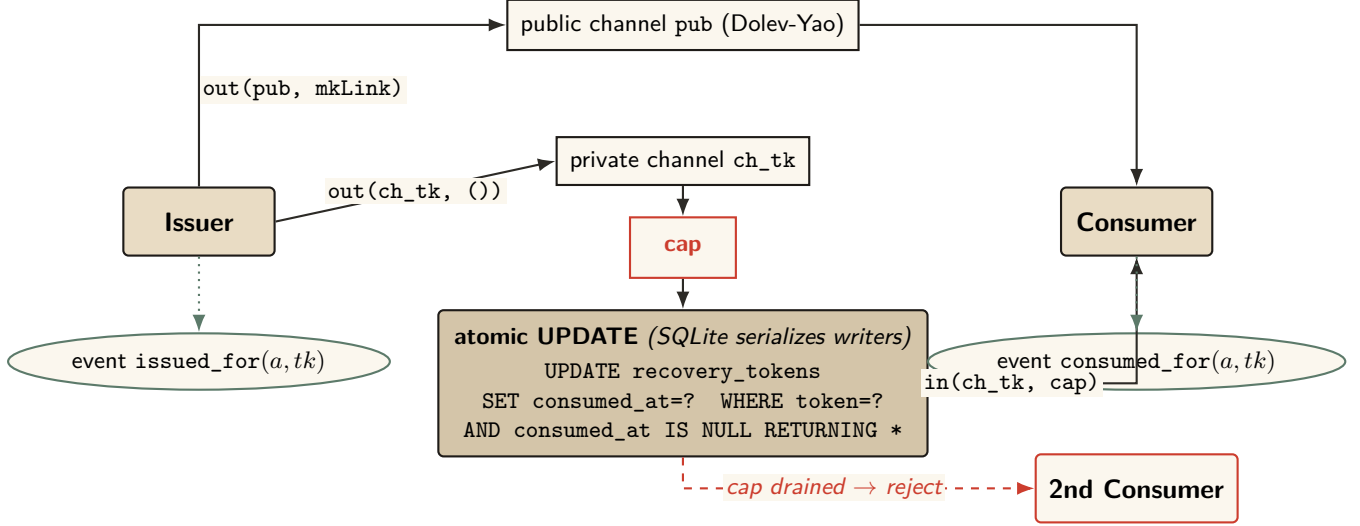


Figure 6: Magic-link single-use atomicity. The SQL UPDATE ...WHERE consumed_at IS NULL RETURNING is the runtime analogue of the ProVerif private-channel cap: SQLite serializes concurrent writers, so the first caller drains the cap and the second consumer's transaction returns zero rows. Verified properties: injective single-use ($\text{inj-event}(\text{consumed_for}) \Rightarrow \text{inj-event}(\text{issued_for})$) and binding.

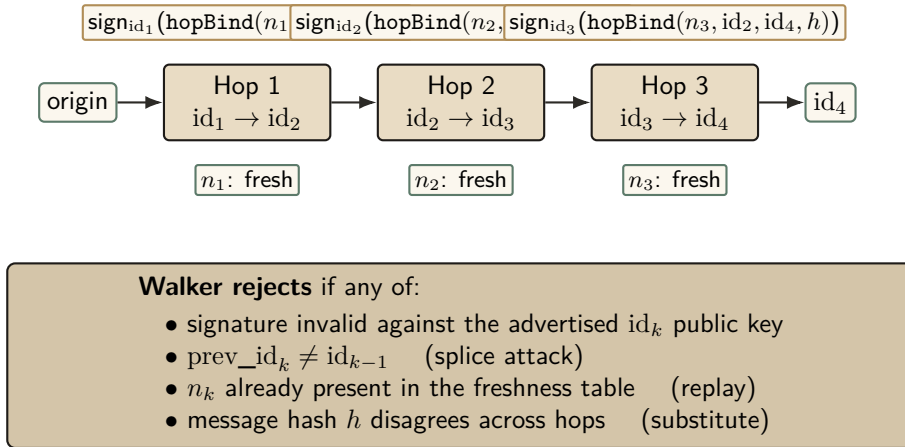


Figure 7: Multi-hop delegation walker. Each hop k signs a binding tuple $(n_k, \text{id}_{k-1}, \text{id}_k, h)$ under Ed25519, where h is the message hash that must remain constant across the chain. The walker verifies depth- N chains with a nonce-freshness table and rejects splices, replays, hop-swaps, and substitution.

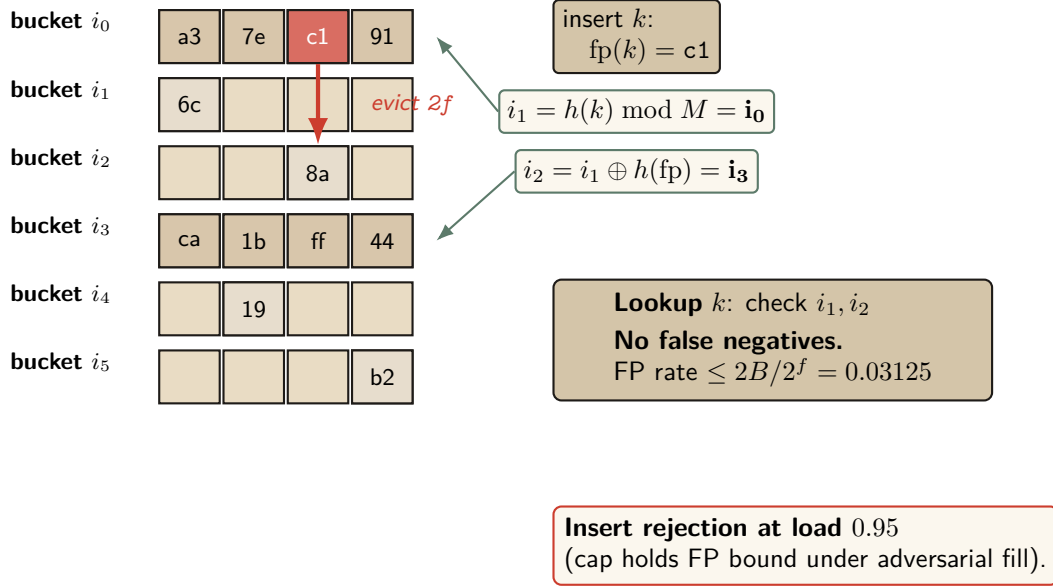


Figure 8: Cuckoo filter (Fan et al. [23]). 8-bit fingerprints, 4 entries per bucket, partial-key cuckoo hashing. Insert lands at either of two candidate buckets $i_1, i_2 = i_1 \oplus h(fp)$; on full, evict and re-insert at the kicked fingerprint’s alternate. The runtime caps load factor at 0.95 to keep occupancy inside the regime where the false-positive bound holds.

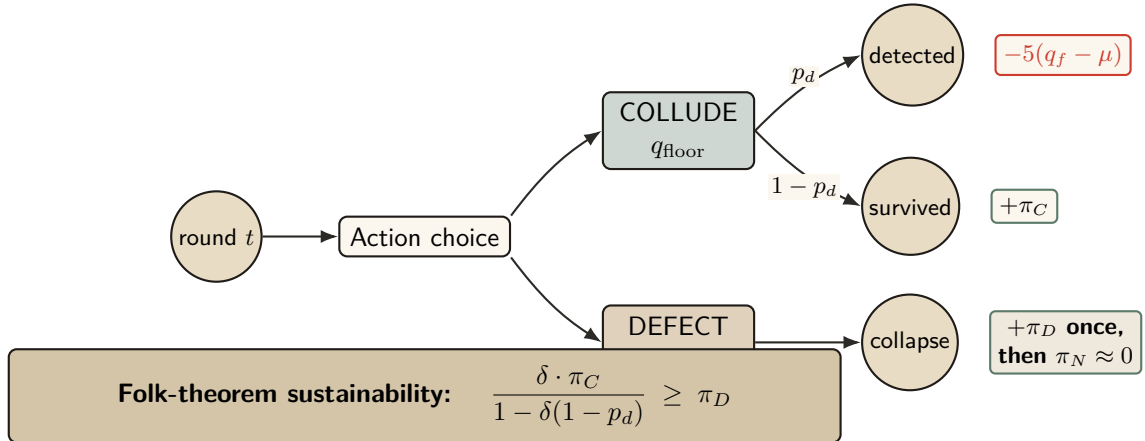


Figure 9: Cartel repeated-game node under the grim-trigger strategy. Each round a colluding member chooses between maintaining the floor q_{floor} or undercutting it by ε . Detection probability p_d slashes the colluder; one-shot defection takes the π_D gain once and forfeits future π_C stream. The sustainability inequality on the right reduces to the closed-form threshold p_d^* used in Figure 10.

C Monte Carlo Visualizations (cartel folk-theorem)

The Pareto-dominance and Sybil-deposit-floor Monte Carlo charts are placed inline next to their respective discussions (Figures 3 and 5 in §7.4.5–§7.4.6). This appendix carries only the A6 repeated-game cartel sweep, whose finer-grain (p_d, δ) grid is supplementary to the body discussion. Raw run logs and the rendering script are bundled at <https://portdaddy.dev/whitepaper/artifacts/>.

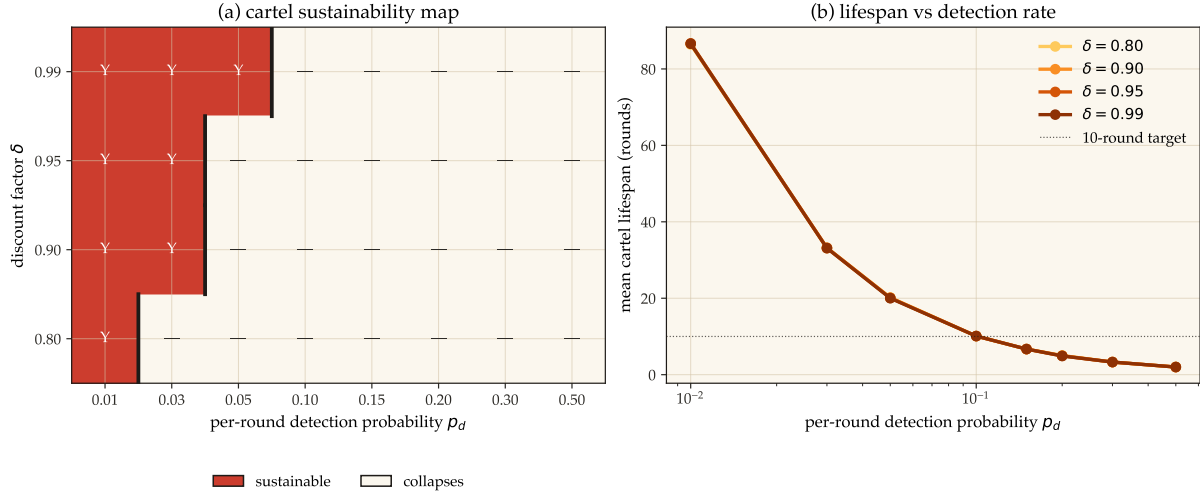


Figure 10: A6 — Repeated-game cartel folk-theorem. Left: sustainability map over the (p_d, δ) grid; red cells are configurations where the grim-trigger cartel beats one-shot defection in expected PV (sustainable), white cells are where it does not (collapses). The thick black vertical bars mark the per-row threshold p_d^* . Right: mean cartel lifespan (rounds until first detection) vs. p_d , one line per discount factor δ . At the realistic $\delta = 0.95$, $p_d^* \approx 0.10$: protocols must achieve $\geq 10\%$ per-round detection to break repeated-game cartels.

D Worked Example: One Agent, All Layers

To make the three-layer architecture concrete, this appendix traces one agent—call it α —through a single bounded task: “Fix the login bug in `auth.ts` and regression-test it.” The example exercises capability issuance, advisory claims, the evidence chain, a mid-task crash, successor resumption, and final settlement. All values are illustrative rather than tuned.

The architecture so far has been described layer by layer. The point of this appendix is to watch the layers *compose*: how a single bounded task progresses through the system, where each layer earns its keep, and what the daemon does *not* touch. The example is deliberately mundane—one small bug, one agent, one principal—because the value of the architecture should be legible in the small before it is trusted in the large.

The task. A principal P wants an agent α to fix a login bug in a hypothetical file `auth.ts` and regression-test it. The acceptance criteria are stated up front:

1. the failing test `login.regression.test.ts` passes;
2. no other test regresses;
3. the diff touches only `auth.ts` and the test file.

P estimates the cleanup cost of α ’s defection or sabotage at \$5—one operator-hour at \$5/hr equivalent, since this is a local agent rather than a cloud workload. Figure 11 traces what follows across the three architectural layers.

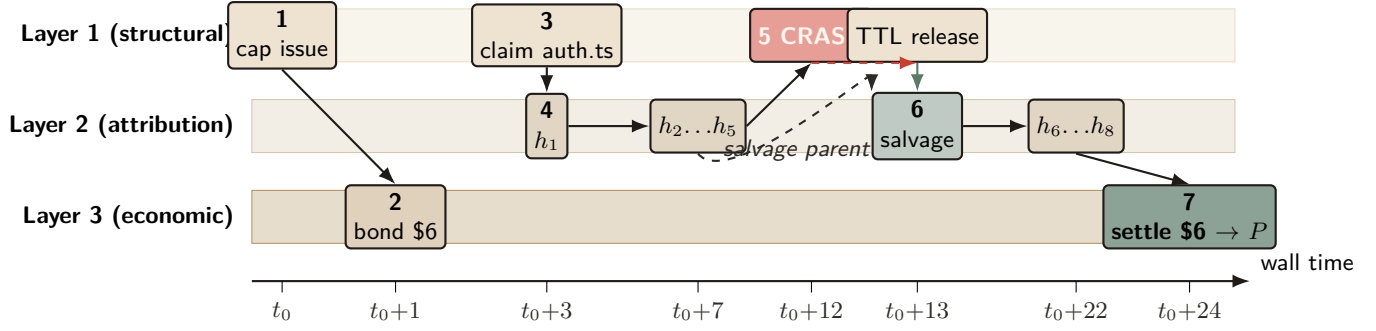


Figure 11: Worked-example timeline (App D). Three swimlanes for the three architectural layers; events placed by wall-clock minute on the x -axis. Layer 1 (structural) issues the capability at t_0 , releases on TTL after crash, and re-issues for the successor. Layer 3 (economic) posts \$6 in escrow at $t_0 + 1$ and settles at $t_0 + 24$. Layer 2 (attribution) accumulates evidence-chain entries h_1 – h_5 on α ’s timeline and h_6 – h_8 on β ’s, linked by an explicit *salvage parent* edge so attribution survives the identity disposal at minute 12.

Phase A: Setup (minutes 0–1)

Before α writes a single byte, two artifacts must exist: a capability that bounds what α can touch, and a bond that prices what α might break.

Step 1: Capability issuance (Layer 1). P requests a Harbor Card for α via the daemon, specifying the capability set $\{\text{fs:read:repo}, \text{fs:write:auth.ts}, \text{fs:write:tests/**}, \text{cmd:test}\}$ and a TTL of 30 minutes. The Anchor Protocol (§3, companion paper [9]) signs the card. The effect is structural rather than advisory: α can prove its identity and capabilities to any verifier, but it *physically* cannot write to, e.g., `database.ts`—attempts return a capability error before the file system is even touched. This is the “physically” that makes the Layer 1 promise non-negotiable.

Step 2: Bond posting (Layer 3). With the capability in place, the Bonded Advisor (§7.4) quotes a bond proportional to the cleanup cost: $B_\alpha = \$5 \cdot (1 + s) = \6 at slope $s = 0.2$. P posts \$6 into escrow. The settlement preconditions are now fixed up front: (1) acceptance criteria met $\Rightarrow$ \$6 returned, (2) partial completion $\Rightarrow$ pro-rata, (3) sabotage $\Rightarrow$ \$6 liquidated to the commons treasury. The bond is the conversion of α ’s “character” into a settled amount; once it is posted, the question of whether α is trustworthy is no longer load-bearing.

Phase B: Work begins (minutes 1–12)

Phase A’s contracts are now in force. α does the work the principal hired it for, and the daemon records what happens.

Step 3: Advisory claim (§5). α claims `auth.ts` via `pd session files add auth.ts`. The daemon’s conflict graph is empty for this path, so the claim succeeds with no conflict notification; had another agent already held the path, α would have been told and would have decided locally whether to coordinate or wait. The claim’s TTL is bounded by the Harbor Card’s—30 minutes—so a crashed α cannot squat on the file indefinitely.

Step 4: Evidence chain begins (Layer 2). α opens the work with a scope note: “*Scope: auth.ts. Hypothesis: cookie-domain handling on subdomain login fails. Validation: npm test login.regression.*” The note’s content hash h_1 chains into the session’s Merkle root $R_{\text{session}}^{(1)}$, and from then on every action α takes—a read receipt for `auth.ts`, a diff blob for the partial

fix, the stdout of the failing test run—is appended as a new entry. Five notes deep, $R_{\text{session}}^{(5)}$ is the canonical commitment to α ’s state of work. The chain is the part of the architecture that survives anything that happens next.

Phase C: Crash and recovery (minutes 12–13)

The crash is where the architecture stops being an abstraction. In Phase B everything went right; in Phase C something goes wrong, and the layers either earn their keep or they don’t.

Step 5: Mid-task crash (§6). At minute 12, α ’s process crashes—let’s say the LLM provider timed out. The daemon’s heartbeat monitor observes α as unresponsive after 90 seconds and marks the session abandoned. α ’s claim on `auth.ts` releases on TTL; the evidence chain is sealed at $R_{\text{session}}^{(5)}$ and committed into the harbor root $R_{\text{harbor},\ell}$ at the next epoch boundary. *Critically, no information is lost:* the partial diff, the test output, and α ’s scope note are all in the chain. The crash terminated a process; it did not terminate the work.

Step 6: Successor agent. P spawns a fresh agent β with a new Harbor Card and the same capability set. β ’s first action is `pd salvage`: the daemon returns the abandoned session, the scope note, the partial diff, and the last test output. β reads the scope note (“cookie-domain handling on subdomain login”), inspects the partial fix that α left behind, and finishes the work. The new evidence chain $R_{\text{session}}^{(\beta)}$ explicitly references $R_{\text{session}}^{(5)}$ as its salvage parent. Attribution is preserved across the identity change (§4.4)—the diff that ships at minute 24 is traceable back to the same principal P that posted the bond at minute 1, even though the agent that did most of the work is gone.

Phase D: Settlement (minute 24)

The work is done. The daemon now decides what was actually accomplished and acts on the settlement preconditions fixed in Phase A.

Step 7: Settlement. β ’s final action is `pd done`. The daemon’s acceptance-criteria checker runs the regression suite, confirms `login.regression.test.ts` passes, confirms no other test regresses, and confirms the diff is bounded to the two files declared in the Harbor Card. All three criteria met. The daemon releases \$6 from escrow to P , signs a settlement record over the evidence-chain root, and appends that record to the harbor Merkle forest (§4.2). The settlement is now itself an immutable, attestable artifact.

What did each layer do?

- *Layer 1 (structural)* kept the agent inside the declared blast radius. The crash, the recovery, and the settlement all happened inside a capability-bounded scope. A capability error at minute 7 would have been a fast-fail, not a corrupted repository.
- *Layer 2 (attribution)* kept the work durable across the crash. Without the Merkle-chained evidence, β would have started from zero; with it, β resumed at minute 12’s state.
- *Layer 3 (economic)* kept the principal honest in scoping. P posted real collateral and recovered it on completion; had α (or β) sabotaged, the cleanup cost would have been pre-funded.

What did *not* happen. The daemon never decided which agent should hold the file, never decided whether the fix was “good enough” in a subjective sense, and never punished the crash. It enforced what *cannot* be (Layer 1), recorded what *did* happen (Layer 2), and settled against the published criteria (Layer 3). The substantive judgments—“is this scope right”, “is this

fix correct”—remained with P and the acceptance-criteria checker. This is the Sen-grounded posture of §5 made operational.

E Exercises

These exercises are for instructors using this paper in a systems, mechanism-design, or multi-agent class. Solutions are not provided; the exercises are designed so that defending an answer requires engaging the paper’s claims rather than recalling them.

E1 (Definitions). Define “advisory claim” and explain, in two sentences, why an advisory regime is strictly more permissive than an enforced-lock regime. Then explain why the Sen impossibility (§5) makes that strict permissiveness a *feature*, not a bug.

E2 (Mechanism design). A reader proposes: “Eliminate bonds. Use post-hoc reputation slashing instead—an agent that defects loses reputation points and is excluded from future work.” Identify two specific failure modes of this proposal that the bonded design avoids. Hint: consider the first-mover problem and Sybil identities.

E3 (Formal verification). Sketch the ProVerif process for the magic-link recovery protocol (companion paper [9], Phase 2). What is the secrecy property you would prove? What is the authentication property? Identify one attacker capability ProVerif would need to model that is not present in the Phase 1 Harbor Card model.

E4 (Coverage-bound A5). The A5 finding (§7.4.6) shows that pure deposit deterrence is bounded by the auction coverage $B_T = \mu(1 + s)$. Construct an attacker who exploits this bound directly. Then construct a defense—other than “raise the deposit”—that defeats the attacker. Hint: see the composite mechanism $C_{kyc} + \text{reputation gating} + B_{dep}^{\text{per-class}}$.

E5 (Cartel folk-theorem). Using the A6 grid (§C, Figure 10), compute the minimum per-round detection probability p_d^* at $\delta = 0.99$. Then argue, qualitatively, what would change if cartel members can re-enter under fresh identities. Does the bond mechanism prevent re-entry, deter it, or merely price it?

E6 (Capability attenuation). An agent α holds a Harbor Card with `fs:write:auth.ts`, `db:read`, TTL = 15 minutes. α delegates to β a card with `fs:write:auth.ts`, TTL = 20 minutes. The verifier rejects this delegation. Cite the specific attenuation invariant violated. Now rewrite β ’s requested card to be acceptable.

E7 (Cuckoo-filter pollution). Sketch a pollution attack against a cuckoo filter that does *not* enforce the load-factor ceiling at 0.95. What invariant breaks? Why does the ceiling defeat it? Hint: the FP rate bound in Fan et al. [23] is conditional on the regime.

E8 (Mechanism extension). The competitive-insurance mechanism (§7.4.5) replaces a single bond price with insurer-agent bids. Propose one alternative pricing mechanism (e.g., a second-price auction with reserve, a Walrasian-style market-clearing rule, or a posted-price catalog) and argue whether it would conditionally Pareto-dominate the static escrow under the CC/NC/RP/NS assumptions of §7.4.5. Identify the assumption your alternative most depends on.

E9 (Open problem). The gossip convergence bound (§??, Anchor Protocol) is reproduced from the standard anti-entropy analysis rather than mechanized. Identify the specific theorem from [24] that would need to be ported to a proof assistant. What would change in the analysis if the daemon mesh is *not* fully connected? Hint: consider gossip on a graph with bottleneck cuts.

References

- [1] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651.
- [2] John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689.
- [3] Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. <https://doi.org/10.1086/259614>
- [4] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.
- [5] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.
- [6] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.
- [7] Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991.
- [8] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [9] Erich Owens. The Anchor Protocol: A Formally Verified Control Plane for Local Agent Swarms. Technical White Paper, Curiositech LLC, May 2026.
- [10] Leslie Lamport. *Specifying Systems: The TLA⁺ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [11] Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
- [12] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014.
- [13] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [14] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [15] William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961.

- [16] Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981.
- [17] Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019.
- [18] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016.
- [19] Michael Rothschild and Joseph Stiglitz. Equilibrium in Competitive Insurance Markets: An Essay on the Economics of Imperfect Information. *Quarterly Journal of Economics*, 90(4):629–649, 1976.
- [20] Charles Wilson. A Model of Insurance Markets with Incomplete Information. *Journal of Economic Theory*, 16(2):167–207, 1977.
- [21] Guy Theraulaz and Eric Bonabeau. A Brief History of Stigmergy. *Artificial Life*, 5(2):97–116, 1999.
- [22] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. RFC 6962, IETF, 2013.
- [23] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014.
- [24] Alan Demers et al. Epidemic Algorithms for Replicated Database Maintenance. In *ACM PODC*, 1987.
- [25] Adi Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [26] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.
- [27] Vitalik Buterin and Virgil Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017.
- [28] Ethan Buchman. Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. M.A.Sc. thesis, University of Guelph, 2016.
- [29] Ethereum Foundation. Ethereum 2.0 Phase 0 – The Beacon Chain Specification. <https://github.com/ethereum/consensus-specs>, 2020.
- [30] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph.D. dissertation, Johns Hopkins University, 2006.
- [31] Carl Ellison, Bill Frantz, Butler Lampson, Ron Rivest, Brian Thomas, and Tatu Ylonen. SPKI Certificate Theory. RFC 2693, 1999.
- [32] Kevin Werbach. *The Blockchain and the New Architecture of Trust*. MIT Press, 2018.
- [33] Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum white paper, 2014.